

CSE 719 — Distributed File Systems & AFS Solutions

Lecture-08 · Exam: Wed 1 Jul 2026 · Dr. Atiqur Rahman

Contents

1	Reference: DFS Fundamentals	2
1.1	What is a DFS?	2
1.2	Why UNIX Semantics is Difficult in a DFS	2
1.3	Vanilla DFS: Flat File Service API	2
1.4	DFS Security	3
1.5	NFS (Network File System)	3
1.6	Andrew File System (AFS)	3
2	2020 Q7a / 2021 Q3b — UNIX Semantics Difficulty in DFS (2.5 / 2 marks)	4
3	2020 Q7b / 2021 Q6c — DFS Design Principles / Requirements (4 / 2 marks)	5
4	2020 Q7c — Cache Validation (2.25 marks)	6
5	2021 Q6d / 2022 Q6c — Andrew File System + Architecture (2 marks each)	6
6	2024 Q3b — Security in DFS + Message Ordering Paradigms (3 marks)	7
7	2024 Q4d — Venus, Vice, and AFS Callback Message Loss (3 marks)	8
8	Summary Table	9

Reference: DFS Fundamentals

What is a DFS?

A **Distributed File System (DFS)** stores files on a *server machine*; client machines perform RPCs to the server to read, write, create, and delete files. Clients access DFS files transparently using the same interface as local files.

Three desirable properties (slides):

1. **Transparency** — client uses same API as local (Unix) files; location, replication, migration are invisible.
2. **Concurrent client support** — multiple clients can read/write simultaneously with *one-copy update semantics*: behaviour appears identical to having exactly one non-replicated copy.
3. **Replication** — files are replicated across servers for fault tolerance.

Why UNIX Semantics is Difficult in a DFS

Unix file operations are **stateful** and **non-idempotent**:

Unix feature	Why it breaks in DFS	Consequence
File descriptors	Per-process handles held <i>at server</i> ; server must track open files per client	Server crash → state lost; complex recovery
Read-write pointer	<code>read(fd, buf, n)</code> advances pointer automatically; same call returns different data each time → <i>not idempotent</i>	Cannot safely retry on packet loss (retried read skips bytes)
Write visibility	Unix: <code>write()</code> immediately visible to all readers on same machine	Across network, writes may be buffered → one-copy semantics violated
Open-then-unlink	Unix allows deleting a file while another process has it open; server must keep it alive until last <code>close()</code>	Distributed reference counting across clients is complex
Blocking read()	Blocks until data available; server holding a blocked thread per client does not scale	Thousands of clients = thousands of blocked server threads

Root cause (from slides): “Unix file system operations are neither idempotent nor stateless.” A DFS must be both — idempotent (for at-least-once retry safety) and stateless (for crash recovery without state reconstruction).

Vanilla DFS: Flat File Service API

The flat file service replaces Unix’s stateful, pointer-based API with a **stateless, idempotent** API:

- `Read(file_id, buffer, position, num_bytes)` — reads from *absolute* position; no pointer advancement.

- `Write(file_id, buffer, position, num_bytes)` — writes at absolute position.
- `Create/Delete(file_id), Get_attributes/Set_attributes(file_id, buffer)`.

No file descriptors (no open state to lose on crash). No pointer advancement (same arguments = same result = idempotent). Directory service provides `lookup(dir, name)` to resolve names to `file_id`.

DFS Security

- **Authentication** — verify that a user is who they claim to be (e.g., Kerberos tickets, password challenge).
- **Authorization** — after authentication, verify that the authenticated user has permission to access the file in the requested mode. Two mechanisms:
 - **Access Control Lists (ACLs)**: per file, a list of allowed users and their access modes (r/w/x). Easy to ask “who can access this file?”
 - **Capability Lists**: per user, a list of files they may access and in what mode. Easy to ask “what can this user do?” Each (user, file) pair = one capability.

NFS (Network File System)

NFS (Sun Microsystems, 1980s) is the canonical real-world DFS implementation. Three-layer architecture:

- **Virtual File System (VFS) module** at client: gives transparency. Processes use file descriptors as normal; VFS decides whether to route each call to the local Unix FS or to the NFS client system. Uses *v-nodes* (virtual inodes) — local file $\Rightarrow$ v-node points to local disk (i-node); remote file $\Rightarrow$ v-node contains remote NFS server address.
- **NFS Client system**: integrated with kernel; performs RPCs to NFS server for file operations.
- **NFS Server system**: plays role of both Flat file service + Directory service from Vanilla DFS. Supports *mounting* — `mount /usr/tesla/inventions` into `/usr/edison/my_competitors`; does not copy files, just re-points the directory.

Server caching: NFS server caches recently-accessed blocks in memory (exploits locality of access).

- **Delayed write**: write to memory, flush to disk every ~ 30 s. Fast but *not consistent* (crash before flush $\Rightarrow$ data lost).
- **Write-through**: write to disk before ack-ing client. Consistent but *slow*.

Andrew File System (AFS)

AFS (Carnegie Mellon, 1980s) solves DFS scalability via **whole-file client-side caching**.

Two processes:

- **Vice** — server-side processes on dedicated file servers. Store files on disk, handle authentication, maintain callback lists, notify clients of changes.
- **Venus** — client-side processes on each workstation. Intercept file system calls, manage local disk cache, contact Vice for cache misses, track callbacks.

AFS file service architecture:

1. Client (**Venus**) intercepts `open(file)`.
2. If not in local cache: Venus fetches entire file from Vice server to local disk.
3. Vice gives Venus a **callback promise**: “I will notify you if this file changes.”
4. Subsequent reads/writes served from *local disk* — zero RPCs, no server load.
5. On `close()`: if file was modified, Venus uploads the new version to Vice.
6. Vice sends **callback break** to all other Venus processes that cached the file $\Rightarrow$ they mark cache entry invalid.

Session semantics: A write to a file is not visible to other clients until the writer closes the file and the readers re-open it. This is a deliberate trade-off: better scalability in exchange for slightly weaker consistency than one-copy semantics.

Callback message loss: If the callback break message from Vice to Venus is lost (network failure), Venus retains a stale cached copy. AFS handles this by:

1. When Venus re-contacts Vice (after reconnection), it *validates all cached files* by checking with Vice whether each callback promise is still valid.
2. If not valid (file changed while offline): Venus discards the stale cached copy and fetches the new version on the next access.
3. Venus also treats cached files as stale after a long disconnection — it does not serve cached data indefinitely without revalidating.

2020 Q7a / 2021 Q3b — UNIX Semantics Difficulty in DFS (2.5 / 2 marks)

Question: Why is it difficult to implement UNIX file system semantics in a distributed file system? Give an example.

Answer

UNIX file system operations are **stateful** (file descriptors track per-client open state at the server) and **non-idempotent** (results depend on implicit server-side pointer, not absolute position). Both properties are incompatible with the requirements of a reliable DFS.

Three core difficulties:

1. Statefulness and crash recovery. In Unix, `open()` returns a file descriptor managed by the server. The server must track which files each client has open and where their read-write pointer is. If the server crashes, all per-client state is lost — clients receive errors with no path to recovery. A DFS must be stateless (no per-client state on server) to allow transparent crash recovery.

Example: Client A calls `open("/data/log")` and receives fd 5. Server crashes. Client A sends `read(5, ...)`: the server has no record of fd 5 and cannot serve the request.

2. Non-idempotent operations. `read(fd, buffer, N)` advances the internal file pointer by *N* bytes automatically. Two consecutive calls with identical arguments return different data. In a lossy network, if the server response is lost and the client retries, the retry causes the pointer to advance a second time, skipping bytes permanently. Idempotency (same args $\Rightarrow$ same result) is required for safe at-least-once RPC retry.

Example: Client sends `read(fd, buf, 1024)`. Server reads bytes 0–1023 and advances pointer to 1024. Response is lost. Client retries. Server now reads bytes 1024–2047 (advances to 2048) — bytes 0–1023 are delivered twice, bytes in between are never read.

3. Immediate write visibility (one-copy semantics). Unix guarantees that a `write()` by process A is immediately visible to a concurrent `read()` by process B on the same machine. Achieving this across a network with client-side caching requires either constant cache invalidation or write-through on every byte, which is prohibitively slow.

2020 Q7b / 2021 Q6c — DFS Design Principles / Requirements (4 / 2 marks)

Question: State the general principles for designing a distributed file system / DFS requirements.

Answer

1. **Transparency.** Clients must access DFS files using the *same interface* as local files (same API: open, read, write, close). The location of the file, number of replicas, and server identities are invisible to client code. This includes: location transparency, replication transparency, and migration transparency.
2. **Concurrency support.** Multiple client processes must be able to read and write files simultaneously. The DFS must enforce *one-copy update semantics*: the effect of concurrent access on a replicated file must appear identical to access on a single, non-replicated copy (no divergence between replicas visible to clients).
3. **Replication.** Files should be replicated across multiple servers for fault tolerance. If one server fails, clients can be transparently redirected to another replica without data loss.
4. **Security.** The DFS must enforce both authentication (verifying user identity) and authorization (verifying the authenticated user has the correct access rights to the file). This is implemented via Access Control Lists (per file) or Capability Lists (per user).
5. **Scalability.** The DFS must maintain performance as the number of clients, servers, and files grows. Stateless servers and client-side caching (as in AFS) are key scalability mechanisms.
6. **Stateless servers / Idempotent operations.** Server operations must not require per-client state. Using absolute file positions (not moving pointers) and unique file identifiers (not file descriptors) ensures operations are idempotent and servers can recover from crashes without complex state reconstruction.
7. **Heterogeneity.** DFS should work across different hardware and operating systems. A common interface (e.g., NFS via VFS layer) allows local and remote files to be accessed identically regardless of the underlying platform.

2020 Q7c — Cache Validation (2.25 marks)

Question: How can the cache be validated in a DFS?

Answer

When a DFS client caches file data locally, the cached copy may become stale if another client modifies the file on the server. Cache validation is the process of checking whether a cached copy is still current.

Two approaches:

1. Client-initiated (polling / timestamp comparison). The client attaches a timestamp to each cached block (the time it was last fetched from the server). Before using a cached block, the client sends a validation request to the server: “is this block still current as of time T ?” The server compares T against the file’s last-modified timestamp:

- If the file has not changed since T : server replies “still valid” — client serves from cache.
- If the file has changed: server returns the new version; client updates cache.

NFS uses this approach with a configurable *attribute cache timeout*: cached attributes are assumed valid for a few seconds, after which they must be revalidated.

2. Server-initiated (callbacks). Used by AFS. When the server sends a file to a client, it issues a **callback promise**: “I will tell you if this file changes.” The client may then use the cached copy indefinitely *without contacting the server* until it receives a **callback break** notification. On receiving a callback break, the client marks the cache entry invalid and fetches the new version on next access.

Comparison:

	Client-initiated	Server-initiated (callbacks)
Server load	High (periodic polls per client)	Low (only on file change)
Freshness	Stale within cache timeout	Immediate (callback on change)
Used by	NFS	AFS

2021 Q6d / 2022 Q6c — Andrew File System + Architecture (2 marks each)

Question: What is the Andrew File System? Describe its file service architecture.

What is AFS?

The **Andrew File System (AFS)** is a distributed file system developed at Carnegie Mellon University (1980s). Its defining design choice is **whole-file caching** on client local disks, enabling servers to handle far more clients than NFS (which caches at the block level on the server). AFS scales because once a file is cached locally, subsequent reads and writes produce *zero server RPCs*.

File Service Architecture

AFS uses two types of processes:

Vice (server side):

- Runs on dedicated file server machines.
- Stores all files on local disk; handles authentication and authorization.
- Maintains a *callback list* for each file: the list of Venus processes that currently have the file cached.
- Sends **callback break** messages to all listed Venus processes when a file is modified.

Venus (client side):

- Runs on each client workstation; integrated with the local kernel.
- Intercepts every file system call (**open**, **read**, **write**, **close**).
- On **open**: if file is not in local cache, fetches the *entire file* from Vice to local disk. Requests a callback promise from Vice.
- On **read/write**: serves directly from local disk — no RPC to server.
- On **close**: if file was modified, uploads new version to Vice; Vice sends callback breaks to all other Venus processes that cached the old version.
- On receiving callback break: marks cached copy invalid; will re-fetch on next **open**.

Key property — session semantics: A file modified by client A is not visible to client B until A closes the file (uploads to Vice) and B re-opens it (triggers re-fetch). This is weaker than one-copy Unix semantics but enables far better scalability.

2024 Q3b — Security in DFS + Message Ordering Paradigms (3 marks)

Question: How do you ensure security in a DFS? Briefly remark on message ordering paradigms in distributed systems.

Security in DFS

DFS security has two mandatory layers:

Authentication — verifying that a user is who they claim to be.

- Implementation: shared-secret challenge-response, or a trusted third party (e.g., Kerberos issues time-limited tickets that the DFS server verifies without needing the password).
- Every client request must carry a credential that the server can verify.

Authorization — after identity is verified, confirming the user has permission to access the requested file in the requested mode. Two mechanisms (from slides):

- **Access Control List (ACL):** stored per file. Contains a list of (user, allowed-modes) pairs. Server checks the file's ACL on every access. Easy to ask: "who can access /data/payroll?"
- **Capability List:** stored per user. Contains a list of (file, allowed-modes) pairs. Client presents a capability token; server verifies it. Can be split into per-(user, file) capability tokens. Easy to ask: "what can user alice do?"

Message Ordering Paradigms

In distributed systems, messages sent between processes arrive in unpredictable order. Four multicast ordering levels:

- **FIFO ordering:** messages from the *same sender* are delivered in the order they were sent. Messages from different senders may interleave arbitrarily.
- **Causal ordering:** if message m_1 causally precedes m_2 (via happen-before relation using vector clocks), every process delivers m_1 before m_2 . Causal $\Rightarrow$ FIFO.
- **Total (atomic) ordering:** *all* processes deliver *all* messages in the *same order*, regardless of sender. Required for replicated state machines and active replication.
- **Hybrid (*-Total):** Total + FIFO or Total + Causal for overlapping groups where a node in two groups must see the same order in both.

Stronger ordering = higher coordination cost. Total ordering requires a sequencer or consensus round per message.

2024 Q4d — Venus, Vice, and AFS Callback Message Loss (3 marks)

Question: What are the Venus and Vice processes in AFS? How does AFS deal with the risk of callback message loss?

Venus and Vice

Vice is the server-side process in AFS. Each Vice server:

- Stores files on its local disk and handles all persistent storage.
- Authenticates clients and checks access rights.
- Maintains a *callback list* per file: which Venus clients currently have that file cached.
- Sends a **callback break** to each Venus on the list when a file is modified.

Venus is the client-side process on each workstation. Each Venus:

- Intercepts all kernel file system calls from local processes.
- On **open**: checks local cache first; on miss, fetches the entire file from Vice to local disk and records a *callback promise* (a token indicating the copy is currently valid).
- Serves **read/write** directly from local disk (no server contact).
- On **close** of a modified file: uploads the new version to Vice.
- On receiving callback break from Vice: marks the cache entry invalid.

Handling Callback Message Loss

Problem: If a callback break message from Vice to Venus is lost (due to network failure, partition, or packet drop), Venus believes its cached copy is still valid when it is actually stale.

AFS solution — revalidation on reconnection:

1. **No indefinite trust.** Venus does not assume a callback promise is valid forever. After a disconnection from Vice (network down, server restart), Venus treats the connection as interrupted.

2. **Validate on reconnect.** When Venus re-establishes contact with Vice, it *explicitly validates* every file in its local cache by sending Vice the cached file's version number or timestamp. Vice checks its current version:
 - If unchanged: Vice confirms the callback promise is still valid.
 - If changed: Vice sends the updated file and a new callback promise.
3. **Conservative caching during disconnection.** If Venus cannot contact Vice and is uncertain whether its callbacks are still valid, it re-fetches any file before serving it to a local process, rather than serving a potentially stale cached copy.

Key insight: AFS trades weak consistency (session semantics) for scalability. The callback mechanism makes consistency close to one-copy semantics in the common case, and the revalidation-on-reconnect protocol handles the failure case without requiring expensive keep-alive polling in normal operation.

Summary Table

Year	Q	Marks	Core answer
2020	Q7a	2.5	UNIX: stateful (fd) + non-idempotent (pointer) → breaks in DFS; retry problem
2020	Q7b	4	7 principles: transparency, concurrency, replication, security, scalability, statelessness
2020	Q7c	2.25	Client-initiated (timestamp poll) vs server-initiated (AFS callbacks)
2021	Q3b	2	UNIX semantics: stateful + non-idempotent (same as Q7a, shorter)
2021	Q6c	2	DFS requirements: transparency, concurrency, replication, security, scalability
2021	Q6d	2	AFS = whole-file caching; Vice (server) + Venus (client); callbacks
2022	Q6c	2	AFS + architecture: Vice stores/callbacks; Venus caches/intercepts; session semantics
2024	Q3b	3	Security: authentication + ACL vs Capability; ordering: FIFO/Causal/Total/Happened-before
2024	Q4d	3	Vice = server (stores, callbacks); Venus = client (caches, intercepts); loss ⇒ revalidation
Total		22.75	

Four facts to know cold:

1. **UNIX is not idempotent and not stateless** — these two properties are why UNIX semantics is hard in a DFS.
2. **AFS = Vice (server) + Venus (client) + whole-file caching + callback promises.**
3. **ACL = per-file list of users; Capability = per-user list of files.**
4. **Callback loss** → Venus revalidates all cached files when contact with Vice is re-established.